

PIMPRI CHINCHWAD EDUCATION TRUST'S PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

*NBA Accredited | NAAC Accredited with 'A' Grade | An Autonomous Institute
AICTE Approved | Permanently Affiliated to SPPU, Pune*

Department of Information Technology

Academic Year: 2026-2027

Assignment No. 1: Merge Sort

Name	: Aryan Ligade
PRN	: 124B1F155
Branch	: Information Technology (IT)
Semester	: V
Batch	: A4
Course Code	: BIT25PC04
Course Name	: Design & Analysis of Algorithms Laboratory
Guided By	: Dr. Rajesh N. Phursule
GitHub Repository	:
Assignment Link	:

Assignment No. 1: Merge Sort

1. Problem Statement:

Design and implement a sorting algorithm using Merge Sort to efficiently arrange customer orders based on their timestamps. The solution should handle a large dataset (up to 1 million orders) with minimal computational overhead. Additionally, analyze the time complexity and compare it with traditional sorting techniques.

2. Course Objective:

- 2.1. To know the basics of computational complexity of various algorithms.
- 2.2. To select appropriate algorithm design strategies to solve real-world problems.

3. Course Outcome:

- 3.1. Analyze the asymptotic performance of algorithms.
- 3.2. Solve computational problems by applying suitable paradigms of Divide and Conquer or Greedy methodologies.

4. Theory:

Merge Sort is a **comparison-based sorting technique** that arranges elements in ascending or descending order by repeatedly breaking a large problem into smaller subproblems. It is based on the **Divide and Conquer** strategy and is particularly useful when dealing with large datasets. Unlike simple sorting methods such as Bubble Sort or Selection Sort, Merge Sort maintains a predictable performance even when the input data is already sorted, randomly arranged, or arranged in reverse order. Another important property of Merge Sort is that it is a **stable sorting algorithm**, meaning that elements having equal values retain their original relative order. The main idea behind Merge Sort is to first create small sorted portions of the original array and then combine those portions in the correct order. The algorithm continues this process until the complete array becomes sorted.

Working of Merge Sort:

- **Divide:** Keep dividing the list or array into two equal halves again and again, until each part has only one element left.
- **Conquer:** Each sub array is sorted individually using the merge sort algorithm.
- **Merge:** The sorted sub arrays are merged back together in sorted order. The process continues until all elements from both sub arrays have been merged.

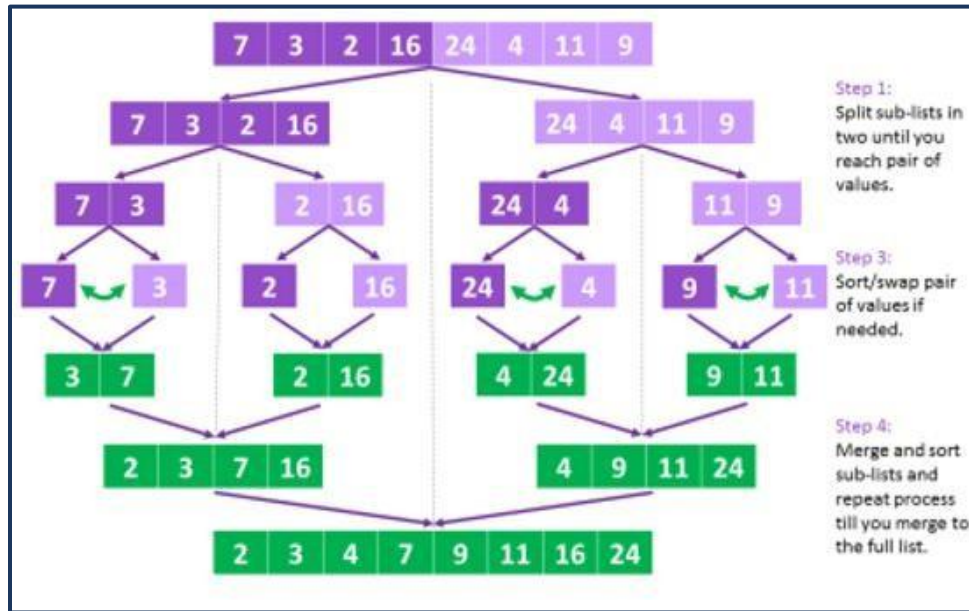


Figure 4.1: Divide and Conquer Execution Tree of Merge Sort

Complexity Analysis of Merge Sort:

- **Divide step:** The array is recursively divided into two halves until each subarray has one element. Number of times we can divide an array of size $n = \log_2(n)$ (height of recursion tree).
- **Merge step:** At each level of recursion, all n elements are merged.
- **Total work:** = number of levels $\times$ work per level = $\log_2(n) \times n = O(n \log n)$.

Analysis of Merge Sort Time Complexity:

- **Best Case Time Complexity:** Even if the array is already sorted, Merge Sort still divides and merges every element. Merging sorted sub arrays still requires scanning through all elements. Best Case = $O(n \log n)$
- **Average Case Time Complexity:** For randomly arranged data, the algorithm still divides into halves and merges them. On average, the number of comparisons during merging is proportional to n . Average Case = $O(n \log n)$
- **Worst Case Complexity:** Even if the array is sorted in descending order (worst arrangement for comparisons), Merge Sort still performs the same divide-and-merge procedure. Comparisons may be maximum, but still bounded by $O(n \log n)$. Worst Case = $O(n \log n)$

Analysis of Merge Sort Space Complexity:

In merge sort, all elements are copied into an auxiliary array of size N , where N is the number of elements present in the unsorted array. Hence, the space complexity for Merge Sort is $O(N)$.

Merge Sort Algorithm:

- **Step 1:** If it is only one element in the list, consider it already sorted, so return.
- **Step 2:** Divide the list recursively into two halves until it can't no more be divided.
- **Step 3:** Merge the smaller lists into new list in sorted order.

Pseudo code:

```

procedure mergesort( var a as array )
  if ( n == 1 ) return a
  var l1 as array = a[0] ... a[n/2]
  var l2 as array = a[n/2+1] ... a[n]
  l1 = mergesort( l1 )
  l2 = mergesort( l2 )
  return merge( l1, l2 )
end procedure

procedure merge( var a as array, var b as array )
  var c as array
  while ( a and b have elements )
    if ( a[0] > b[0] )
      add b[0] to the end of c
      remove b[0] from b
    else
      add a[0] to the end of c
      remove a[0] from a
    end if
  end while
  while ( a has elements )
    add a[0] to the end of c
    remove a[0] from a
  end while
  while ( b has elements )
    add b[0] to the end of c
    remove b[0] from b
  end while
  return c
end procedure

```

5. Flow Diagram:

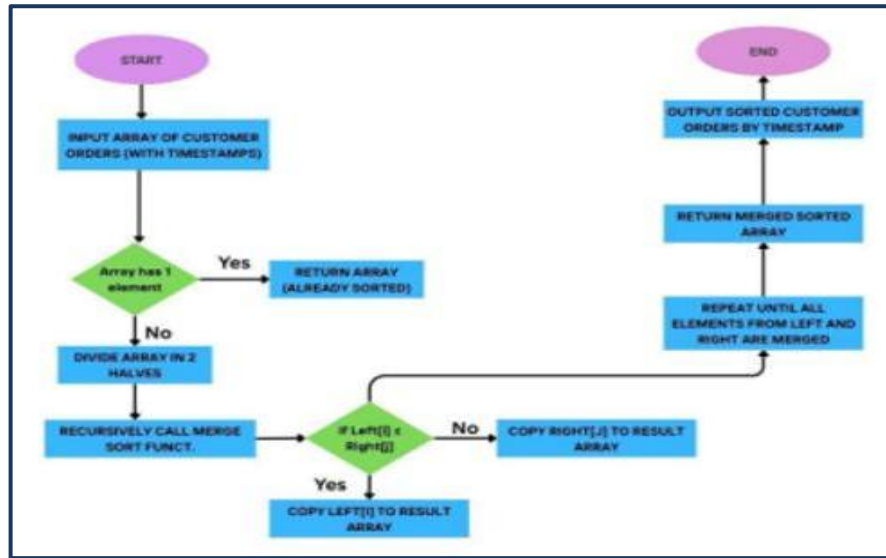


Figure 5.1: Flow Diagram of Customer Order Merge Sort Algorithm

6. Example:

Customer order timestamps: [7, 3, 2, 16, 24, 4, 11, 9]

• Step 1: Divide

- Split into halves → [7, 3, 2, 16] and [24, 4, 11, 9]
- Further split into pairs → [7, 3], [2, 16] and [24, 4], [11, 9]
- Split into individual elements → [7], [3], [2], [16], [24], [4], [11], [9]

• Step 2: Merge & Sort

- Merge [7] and [3] → [3, 7]; Merge [2] and [16] → [2, 16]
- Merge [24] and [4] → [4, 24]; Merge [11] and [9] → [9, 11]
- Merge [3, 7] and [2, 16] → [2, 3, 7, 16]
- Merge [4, 24] and [9, 11] → [4, 9, 11, 24]
- Merge final two halves → [2, 3, 4, 7, 9, 11, 16, 24]

Output:

Orders sorted by timestamp → [2, 3, 4, 7, 9, 11, 16, 24]

7.Code and Output:

Repository:

Assignment :

8.Conclusion:

Merge Sort provides an efficient and systematic way of arranging data by breaking a large sorting problem into smaller and easier subproblems. The individual portions are sorted and then combined carefully to produce the final ordered sequence. Its divide-and-conquer approach gives a consistent time complexity of **$O(n \log n)$** for best, average, and worst cases, making it suitable for handling large datasets. Although the algorithm requires additional memory for the merging process, its stability and predictable performance make it an important sorting technique in computer science and practical data-processing applications.